

SciForge: A Multimodal Research-Native AI Workbench for Scientific Discovery

Author Names^{1*}

^{1*}Institute, University, City, Country.

Corresponding author(s). E-mail(s): author@example.com;

Abstract

Scientific work increasingly spans heterogeneous artifacts—papers, code, datasets, scientific file formats, model outputs, figures, manuscripts, and team decisions—yet general-purpose AI assistants rarely preserve these objects as a coherent, auditable research state. We present SciForge, a multimodal research-native AI workbench that reserves the graphical interface for human judgment while search, parsing, model routing, workflow execution, plotting, writing, and presentation generation run as modular agent-accessible services. SciForge is built around four integrated pillars: (i) *translate-then-reason* for **multimodal** scientific ingress, routing native scientific objects through domain translators before the agent reasons; (ii) *evidence governance* for **auditable** traceability, linking claims to provenance chains with structured audit; (iii) *collaborative team science* for **collaborative** research, enabling multi-role decision governance with shared review surfaces; and (iv) *practical scientific impact*, validated through end-to-end workflows spanning BGC discovery, AI-guided protein and molecular design, automated figure generation, and multi-format document parsing. The system combines a thin interaction layer, contextual research capability patterns, an Agent Runtime and Workflow Engine, an Evidence-DAG audit sidecar, a Scientific Model Router, and local scientific memory backed by reusable worker services. SciForge currently runs as a desktop application, with mobile supervision support; future releases will deepen team collaboration through shared review surfaces and decision governance. The system is open-source and available at <https://github.com/AGI4Sci/SciForge>.

Keywords: SciForge, AI workbench, scientific discovery, agentic workflow, evidence-aware AI, multimodal science, research automation

1 Introduction

1.1 Background and Motivation

Scientific discovery is becoming increasingly computational, multimodal, and collaborative. A single research project may move among scholarly papers, experimental protocols, command-line tools, notebooks, gene and variant files, protein sequences, three-dimensional structures, molecular records, single-cell matrices, generated figures, manuscript drafts, slide decks, and group decisions. These objects are not independent files. A manuscript claim may depend on a raw dataset, a preprocessing script, a model run, a visualization parameter, a literature passage, and a later human decision about whether the evidence is strong enough to use. As a result, the central bottleneck is no longer only access to models or tools, but the lack of a persistent environment that can organize scientific objects, agent actions, human decisions, and evidence traces as one coherent research state.

Recent AI systems have demonstrated impressive capabilities for literature search, hypothesis generation, coding, experimentation, and scientific writing [9–12, 19]. At the same time, desktop and terminal-capable scientific workbenches show the importance of keeping execution close to local files, remote servers, and HPC environments [4, 6, 13, 16, 20]. However, most existing systems still emphasize either task-specific intelligence, conversational assistance, or local execution surfaces. They rarely treat the full research process as a **goal-oriented**, **multimodal**, **auditable**, and **collaborative** workflow in which scientific data, agent reasoning, generated artifacts, and manuscript claims remain connected over time.

This gap matters because scientific AI agents must satisfy four interconnected demands absent from general-purpose assistants. **Goal-oriented** research design requires persistent objectives, review gates, and shared decision records spanning turns, tools, and collaborators—not session-scoped conversations. **Multimodal** inputs demand native scientific file handling: FASTA sequences, PDB/mmCIF structures, SMILES strings, variant files, and cell-omics tables cannot be directly prompted without domain-specific translation. **Auditable** evidence traces must link every conclusion to its supporting data, script, model, parameter, paper, or expert translation, and flag when figures or actions require human-PI approval. **Collaborative** governance enables team decisions, goal-scoped approvals, cross-session handoffs, and shared review of claims and evidence across multiple researchers. Section 3 shows how SciForge’s architecture—the Scientific Model Router for multimodal translate-then-reason, the Evidence DAG and Project DAG for auditable provenance, and multi-role decision governance for collaborative science—addresses each demand structurally rather than through ad-hoc prompting.

We present *SciForge*, a multimodal research-native AI workbench for scientific discovery. SciForge is designed as a local-first operating environment for research rather than a wrapper around a single model. Its architecture organizes interaction surfaces, research capability patterns, a core engine with evidence governance, a scientific model router, and local-first infrastructure into a coherent layered workbench; Section 3 describes each layer in detail.

1.2 Contributions

SciForge’s core scientific contributions address four fundamental gaps in current research AI systems:

- **Multimodal Ingress.** End-to-end scientific file processing via translate-then-reason: four classes of native scientific objects—protein sequences (.faa; .fasta/.fa), protein structures (.pdb/.cif/.mmcif), molecules (.smi/.smiles), and single-cell transcriptomics (via Cell2Sentence [18])—are routed through domain-expert translators (Esm2Text, Prot2Text [1], BioT5 [17], C2S) that return structured expert observations before the main agent reasons. Translator output is an evidence candidate, not a verified fact; the human remains responsible for judgment.
- **Evidence Governance.** Every agent action—file read, model call, tool invocation, script execution—attaches structured provenance (software version, parameters, environment, random seed) to a thread-level Evidence Snapshot. A read-only audit sidecar inspects these snapshots and surfaces potential gaps without blocking the agent’s work, linking claims to concrete data, scripts, models, and parameters. A goal-scoped Project DAG organizes research goals, evidence snapshots, and review decisions into a shared, auditable structure supporting candidate and certified release gates.
- **Collaborative Science.** SciForge supports multi-role, collaborative research where reviewer/approver roles, IM-based coordination channels (Zulip, Discord, WeChat, Feishu), and shared review surfaces enable cross-session handoffs and team-level decision governance. The Project DAG makes collaborative review workflows auditable: researchers inspect Evidence Snapshots, record role-attributed ReviewItems, and advance artifacts through candidate/certified release gates with traceable human-PI oversight.
- **Practical Impact.** SciForge is validated through end-to-end scientific workflows spanning biosynthetic gene cluster (BGC) discovery, AI-guided protein and molecular design with experimental evaluation criteria, automated journal-style figure generation, multi-format document parsing, and AI-assisted presentation generation from research artifacts—each with explicit evidence trails, audit findings, and governance checkpoints documented in Section 4.

The SciForge codebase, including the Evidence-DAG audit worker, decision-record contracts, agentic skills, and scientific model router, is open source at <https://github.com/AGI4Sci/SciForge>.

2 Related Work

2.1 GUI-Based Scientific Workbenches

GUI-based scientific workbenches, including desktop and web applications, make AI assistance accessible through human-facing workspaces. Claude Science is the closest broad commercial reference: it is described as an AI workbench for scientists with desktop, local, SSH, and HPC-facing execution, scientific skills, specialist agents, generated artifacts, and reviewer agents for citations and calculations [4]. ClaudePrism

focuses on the manuscript side of the workflow, combining a native desktop environment with local LaTeX compilation, Python environments, Claude Code sessions, and scientific writing skills [6]. Web tools such as Elicit and Consensus provide polished interfaces for literature search, synthesis, and research-agent style review [7, 8]. These systems demonstrate the importance of usable research interfaces, but they are usually centered on one model family, one workflow domain, or one user session. SciForge instead treats the GUI as a thin layer for human judgment over a local, modular, multi-agent research substrate.

2.2 Terminal Workbenches

Terminal-oriented workbenches prioritize execution close to local files, remote servers, and HPC environments. OmicOS provides a Rust `omicos-core` binary that can run as a local daemon serving a browser UI or as `omicos cli`, with analysis code running in a shared IPython kernel and raw data remaining local [13–15]. OmicsClaw follows a local-first multi-omics design with CLI, TUI, desktop, web backend, and SSH remote execution surfaces over a shared agent loop [16]. Operon targets bioinformatics workflows across desktop and remote cluster environments, using persistent terminal sessions so jobs execute where data and schedulers live [20]. These systems are strong at local and remote execution, but they generally expose tools and workflows rather than a unified layer for multimodal scientific routing, research decision surfaces, team supervision, and manuscript-to-artifact continuity.

2.3 API and Research Agents

API-based and research-prototype agents explore more autonomous scientific reasoning. FutureHouse exposes agents for literature search, deep review, precedent search, and chemistry planning through a web/API platform [10]. Google’s AI Co-Scientist studies multi-agent hypothesis generation, debate, and ranking for biomedical research [11]. The AI Scientist and Agent Laboratory automate parts of the machine-learning research loop, including ideation, coding, experimentation, paper drafting, and review [12, 19]. Literature systems such as PaperQA2, OpenScholar, and ScholarQA focus on retrieval-augmented scientific question answering and literature synthesis [2, 5, 9]. These systems demonstrate the power of specialized agents, but they are less focused on being a persistent local workbench where data, code, figures, decisions, and generated artifacts remain in one workspace.

2.4 Comparison and Positioning

While each of the three system categories addresses a distinct facet of the AI-assisted research life cycle, SciForge is designed to unify them within a single workbench. The comparison spans five key dimensions: execution, model governance, evidence quality, research continuity, and goal-oriented research design.

Execution architecture. GUI workbenches such as Claude Science and Claude-Prism deliver polished interactive experiences but are typically constrained to one model provider and one local session. Terminal workbenches—OmicOS with its hybrid daemon plus browser model, OmicsClaw with multi-surface CLIs, and Operon with

persistent terminal sessions—excel at running code where data and schedulers reside, yet they expose tools and pipelines rather than a unified research workbench. SciForge bridges these worlds by providing a GUI for human judgment while keeping a local-first workspace where execution can span desktop, SSH, and HPC environments.

Model governance. The reviewed public materials commonly emphasize a single provider family or manual switching between separate tools. SciForge’s Scientific Model Router combines multi-provider routing with a deliberately narrow translate-then-reason file-ingress path covering four modality classes: protein sequences (Esm2Text [1]), protein structures (Prot2Text [1]), molecules (BioT5 [17]), and single-cell transcriptomics (Cell2Sentence [18]); unsupported formats fail closed. Literature is handled through search, PDF anchoring, and source attribution, not as a scientific modality translator. We did not find this same integrated combination documented as a first-class feature in the reviewed categories.

Evidence quality and governance. Research agents such as FutureHouse, AI Co-Scientist, and the AI Scientist produce outputs from variable-quality evidence; in the publicly documented materials we reviewed, we did not observe the same integrated combination of claim–evidence audit trail, PROV-JSON provenance, and quality gates in a single research workbench. SciForge threads claims through an Evidence DAG backed by PROV-JSON provenance, audit runs, risk digests, and explicit quality gates. This governance layer is designed to make agent outputs inspectable and auditable.

Research continuity. Many reviewed systems foreground per-session interaction. SciForge maintains goal-centered research memory as scoped free-text records (scope, tags, confidence, timestamps, context) across sessions; typed goals and decision events belong to the Project DAG, enabling long-running multi-session scientific workflows rather than only single-turn conversations. The combination of research memory, workflow engine, and evidence governance creates a persistent research workspace whose integrated design—tying thread-level Evidence Snapshots to goal-scoped Project Snapshots with audit and release semantics—we did not find documented as a first-class, integrated feature in the public materials of the reviewed GUI, terminal, and API-agent systems.

Goal-oriented research design. SciForge anchors every thread and project to an explicit goal scope with defined review gates, decision records, and release semantics. This design ensures that scientific objectives are persistent, auditable, and governable across multiple turns, tools, and contributors.

Table 1 summarizes these differences at the category level, while the detailed capability differentiation in Appendix A1 provides a feature-level comparison against common approaches.

SciForge combines these categories rather than competing with only one of them. It keeps the usability of GUI workbenches, the locality and composability of terminal systems, and the task power of scientific agents, while adding fail-closed scientific-object handling, content-addressed artifacts with structured selection and provenance, specialist domain translation, run-grounded evidence governance with immutable snapshots and an audit sidecar, and goal-scoped decision governance with explicit release semantics.

Table 1 High-level comparison of scientific AI system categories. A checkmark indicates first-class or documented support as a primary design goal in the publicly reviewed materials. Absence of a checkmark does not prove absence of a feature; it indicates we did not find documented first-class support in the reviewed materials. † Planned for a future release.

Capability	GUI WB	Terminal WB	API agent	SciForge
Human Interface — GUI for interactive research	✓			✓
Local Execution — local/HPC compute, local-first data		✓		✓
Sci. Tools — fail-closed native format ingestion				✓
Sci. Multimodal Routing — enhanced scientific multimodal understanding				✓
Workflow Automation — DAG-based scheduled protocols				✓
Research Memory — cross-session knowledge persistence				✓
Evidence Governance — provenance, audit trail, release gates				✓
Team Workspace — multi-user collaboration [†]				†

3 System Architecture

3.1 Overall Design Philosophy

SciForge is organized as a layered research operating environment. **Layer 1** provides thin judgment surfaces for desktop, group, and mobile interaction. **Layer 2** encapsulates six research capability patterns—Literature Review, Idea Generation, Experiment Design, Experiment Execution, Analysis & Iteration, and Scientific Communication—that cover the scientific workflow while sharing one evidence-aware control chain. **Layer 3** houses the core engine: agent runtime, workflow automation, scoped memory, and evidence governance. **Layer 4** is the Scientific Model Router that detects modality and dispatches to domain-expert translators. **Layer 5** supplies infrastructure: local memory, modular workers, scientific connectors, and reproducible run substrate.

SciForge follows three implementation principles that distinguish it from feature-heavy desktop software:

1. **Thin GUI.** The interface appears only when human judgment is needed—reviewing, annotating, approving, comparing; all other capabilities run in the background agent runtime.
2. **Modular services.** Search, parsing, plotting, model routing, workflow execution, and presentation generation run as MCP-compatible tools or worker processes, keeping the application close to a thin interaction shell [3].

3. **Local-first evidence model.** Data, traces, artifacts, and evidence graphs remain anchored to the research workspace; remote collaboration and mobile access are control channels, not replacements for local state.

Figure 1 summarizes the layered architecture.

3.2 User Interaction and Collaboration (Layer 1)

Layer 1 exposes three interaction surfaces driven by the same underlying runtime. The **desktop application** provides a thin GUI reserved for human judgment—reviewing evidence, annotating figures, comparing candidate artifacts, and approving release decisions—while execution runs in the background. **Group chat integration** (Zulip, Discord, WeChat, Feishu) enables remote task submission, status polling, and lightweight approvals without switching context. **Mobile supervision** enables remote monitoring and intervention for long-running experiments, with task submission and review actions gated through the same governance model.

3.3 Research Capability Patterns (Layer 2)

SciForge supports six research capability patterns with explicit scientific input–output boundaries. *Literature Review* maps papers and source records to a field map that summarizes prior work and the current state of a domain. *Idea Generation* maps existing evidence to research questions, hypotheses, and candidate methods. *Experiment Design* turns a selected idea into a test plan with validation procedures, metrics, controls, and success criteria. *Experiment Execution* maps that plan to run records and artifacts by executing code, conducting experiments, invoking instruments, or delegating to computational pipelines. *Analysis & Iteration* maps results to diagnoses, revised assumptions, and a justified next step. *Scientific Communication* maps reviewed evidence and research artifacts to papers, reports, figures, and presentations. These capability patterns cover a common scientific workflow but are not a rigid linear pipeline: researchers may enter at any stage and iterate between stages. They remain contextual patterns within one workbench rather than independent applications with separate execution semantics. Regardless of entry point, each task follows the same evidence-aware control chain: (1) task and object ingress; (2) fail-closed translation when a native scientific object requires a domain expert; (3) delegated agent and worker execution through Layers 3 and 4; (4) provenance-attached evidence candidates with artifact and run lineage; (5) a thread-scoped Evidence Snapshot and advisory audit; and (6) goal-scoped review and candidate/certified release through the Project DAG and human PI.

3.4 Core Engine (Layer 3)

The Core Engine plans and executes research work through coordinated subsystems.

SciForge uses two connected evidence layers for scientific governance. The **Evidence DAG** (thread-scoped) automatically constructs a claim–source–reasoning graph from each agent session’s completed turns and persists the graph as PROV-JSON. The **Project DAG** (goal-scoped) consumes immutable Evidence DAG snapshots across

SciForge: A Multimodal Research-Native AI Workbench for Scientific Discovery

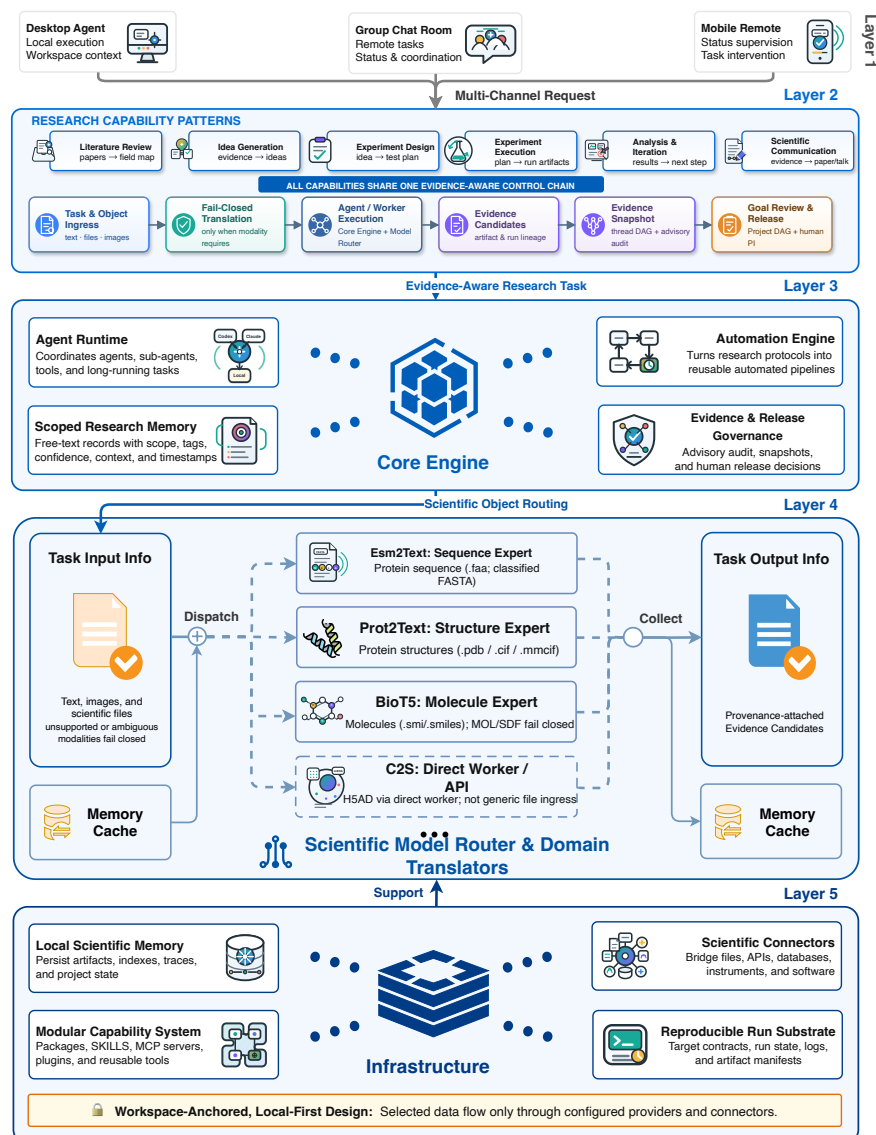


Fig. 1 SciForge system framework. Layer 2 exposes six scientific-workflow capabilities—Literature Review, Idea Generation, Experiment Design, Experiment Execution, Analysis & Iteration, and Scientific Communication. Each capability enters the same evidence-aware control chain—ingress, translation, execution, evidence capture, audit, and release—built on the interaction surfaces, core engine, scientific model router, and local-first infrastructure.

sessions, merges equivalent findings while preserving independent source paths, and links claims to project objectives. Both layers compile asynchronously behind the main agent, keeping evidence up to date without blocking exploration. Audit runs are

read-only sidechains that produce structured risk digests—flagging ungrounded claims, unresolved contradictions, weak support, or low-credibility sources—without mutating DAG state. All decisions, whether human or AI, enter through structured Decision Events, giving every modification a traceable, reviewable record.

The **Agent Runtime** uniformly supports multiple agent backends—Codex, Claude Code, and custom runtimes—allowing research groups to select the execution environment that best matches their scenario: Codex for local-first interactive coding with full workspace access, Claude Code for extended autonomous task chains, and custom runtimes for specialized scientific computing or institutional deployment requirements. Across all backends, the Runtime coordinates the main agent, delegated sub-agents, tool execution, workspace operations, MCP worker invocation, and long-running task recovery through a consistent API surface. The **Workflow Engine** formalizes reusable pipelines with code, agent, tool, and approval nodes; scheduled execution and loop-style fixed protocol automation are treated as run modes rather than separate architectural modules. **Scoped Research Memory** stores free-text records (scope, tags, confidence, timestamps, context) across sessions; typed goals and decision events belong to the Project DAG. **Evidence & Release Governance** exposes goal-scoped release decisions (candidate and certified gates) through the Project DAG without synchronously blocking exploratory execution.

3.5 Scientific Model Router (Layer 4)

The **Scientific Model Router** is a core architectural innovation of SciForge, acting as a universal model invocation plane that abstracts away provider heterogeneity while retaining the full expressiveness of each backend. Rather than forcing a lowest-common-denominator interface, the Router supports three request-response protocols—OpenAI Chat Completions, OpenAI Responses, and Anthropic Messages—selected per provider through a normalized endpoint format, allowing each model to be invoked through its native, most capable API surface.

Intelligent Auto-Routing. The Router includes a lightweight classifier that inspects the user request and recent context to automatically select the appropriate reasoning effort (off, high, or max), with a deterministic heuristic fallback so routing decisions do not rely on a single point of failure.

Provider-Aware Optimization. For DeepSeek models, the Router performs proactive reachability probing, applies native thinking controls, and computes real-time cache-aware cost estimation in both USD and CNY. For all providers, it implements automatic retry with backoff, error classification, and stream-idle timeout detection.

Output Robustness and Repair. The Router repairs malformed tool-call arguments—stripping Markdown fences and recovering partial JSON—and extracts embedded images from tool results as standard model attachments, enabling multimodal reasoning chains across scientific visualizations and model analysis.

Scientific Modality Routing. For scientific files, the Router detects modality-specific extensions and applies one of two handling paths. *Translate-then-reason* applies to four currently supported modalities: protein (.faa; .fasta/.fa with per-record conservative content classification) translated by Esm2Text, protein structures (.pdb, .cif, .mmCIF) by Prot2Text, small molecules (.smi, .smiles) by BioT5, and single-cell

transcriptomics via Cell2Sentence (C2S). Each translator output carries provenance metadata, allowing the main agent to reason over structured expert observations. VCF, BED, GFF, and MGF are unsupported and fail closed; raw content is not routed to the general reasoner.

Multimodal translation caches store scientific and visual observations by content hash, enabling repeated requests to reuse expert evidence without recomputing final conclusions.

3.6 Infrastructure (Layer 5)

The Infrastructure layer provides the durable substrate that anchors research activity to the local workspace. **Local Scientific Memory** stores papers, datasets, parsed documents, scientific file manifests, embeddings, indexes, tool outputs, run metadata, and artifact caches. The **Modular Capability System** packages computational functionality as Skills, MCP servers, Plugins, HTTP services, or worker processes—each independently versioned and upgradeable. **Scientific Connectors** normalize external resources such as papers, datasets, databases, APIs, instruments, notebooks, analysis scripts, reference managers, LIMS/ELN systems, and institutional storage into a unified access layer. The **Reproducible Run Substrate** records execution-target contracts, run and job state, logs, and artifact manifests so that downstream evidence can refer to concrete execution records. SciForge remains workspace-anchored and local-first; selected data may flow through explicitly configured model providers, connectors, or remote execution services.

4 Use Cases

This section uses three evidence-status labels consistently: *[Implemented]* for a product path exercised in SciForge, *[Validated Prototype]* for a bounded single-case or externally archived pilot with stated limitations, and *[Illustrative Future]* for an unexecuted target workflow. These cases are not separate product panels; they show what is implemented, what has only pilot evidence, and what remains a design target on the same modular substrate.

4.1 Agentic Research Sprint: From Question to Manuscript Package

Setting. *[Implemented]* A human PI poses the biological question “Which molecular systems drive mammalian germ cells into meiosis?” and delegates execution to SciForge, an AI4S research workbench. Codex—an AI coding agent replacing the human expert role—orchestrates SciForge’s agent-accessible services—literature search, candidate gene identification, structured hypothesis formulation, parallel sub-agents for multi-omics evidence synthesis, AlphaFold3 structural assessment, and iterative methodology hardening—within a multi-day, PI-controlled agentic loop (Fig. 2), with Codex serving

as the PI's AI delegate. Outputs are reviewed through evidence summaries, reviewer-style self-audits, and human decisions. After 132 stages and 199+ Git-tracked commits, the repository¹ records the control loop as a local workspace log.

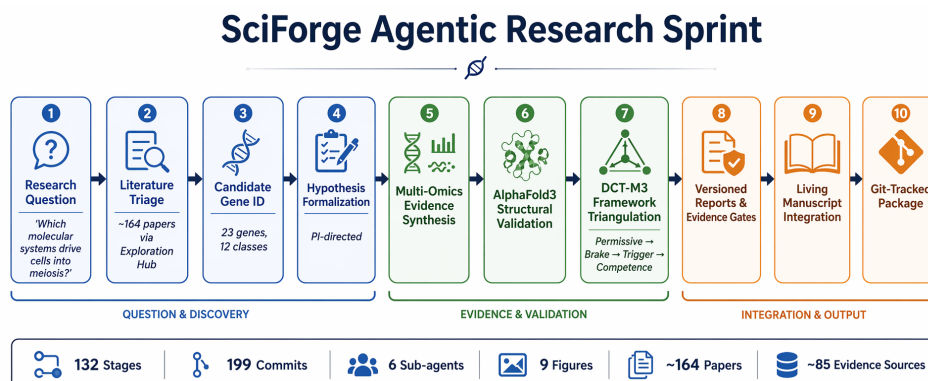


Fig. 2 Automated research pipeline executed during the SciForge scenario-01 research sprint. The 132-stage, 199+-commit agentic loop progresses from a biological research question through literature mining (DCT-M3), candidate gene curation, AlphaFold3 structural prediction, evidence triangulation, manuscript synthesis, and submission packaging. Quantitative metrics: 132 research stages, 199+ Git commits, 6 parallel child-agent runs, 9 generated figures, ~164 papers reviewed, and ~85 evidence nodes produced. Selected artifacts and thread evidence are linked to versioned records, evidence summaries, and audit findings where captured.

Discovery. The systematic agentic pipeline yielded a candidate-gene atlas of 23 genes across 12 molecular classes, with claims traceable to specific evidence sources (Fig. 3). Central to the discovery was the DCT-M3 causal-module triangulation framework, which decomposes the meiosis initiation switch into four regulatory layers: (1) permissive licensing—chromatin opening and epigenetic priming via pioneer transcription factors; (2) brake release—degradation of meiotic inhibitors through ubiquitin-proteasome pathways; (3) trigger execution—SPO11-mediated programmed double-strand breaks and synaptonemal complex assembly; and (4) chromatin/RNA competence—transcriptional activation, alternative splicing, and 3D genome reorganization. Multi-omics evidence (transcriptomics, epigenomics, proteomics) was cross-validated against AlphaFold3 structure-based computational assessments and systematic literature triangulation to ground each candidate in causal evidence. This scenario demonstrates that SciForge can sustain long-horizon, evidence-constrained research—not by auto-generating a paper, but by producing an inspectable, auditable research package whose reasoning and evidence trail remain open to human scrutiny.

Informal Post-Hoc Literature Mapping. A post-hoc review of the 23-gene atlas against the published meiosis literature found that 22 of the 23 genes have known meiosis associations; MAPK fell outside established meiosis paradigms. This retrospective mapping was conducted informally and was not guided by a pre-registered protocol. Formal expert adjudication of gene-level relevance, along with inter-annotator agreement metrics and a pre-registered evaluation protocol, remains future work.

¹<https://github.com/AGI4Sci/scenario-01-research-sprint> (archive available at time of writing)

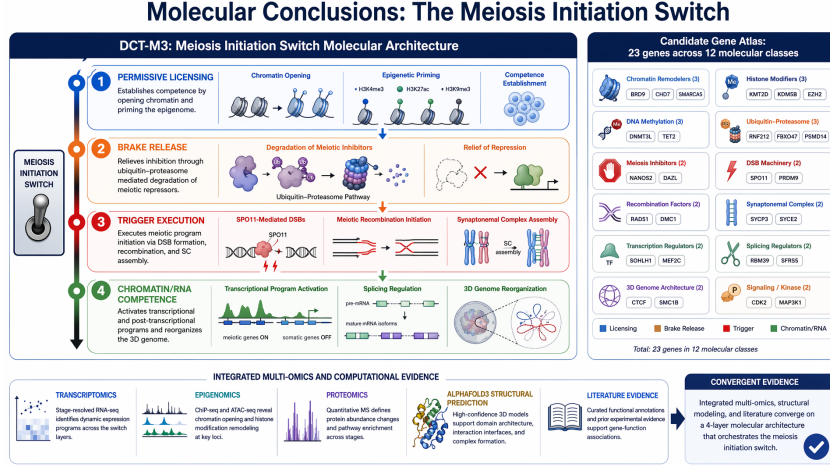


Fig. 3 Key biological conclusions reported by the SciForge agentic research sprint. The DCT-M3 causal-module triangulation framework decomposes the meiosis initiation switch into four regulatory layers: (1) permissive licensing—chromatin opening and epigenetic priming; (2) brake release—degradation of meiotic inhibitors via ubiquitin-proteasome pathways; (3) trigger execution—SPO11-mediated double-strand breaks and synaptonemal complex assembly; (4) chromatin/RNA competence—transcriptional activation, splicing regulation, and 3D genome reorganization. The sprint reported a candidate-gene atlas of 23 genes across 12 molecular classes, supported by multi-omics evidence, AlphaFold3 structure-based computational assessment, and literature triangulation; formal expert adjudication remains future work.

4.2 AI4AI: Automated Modeling for Scientific Discovery

[Validated Prototype] AI-for-AI (AI4AI) refers to a design paradigm in which agents read, distill, and recombine algorithmic building blocks from existing codebases into new machine learning models. Rather than treating model design as a one-shot prompt, AI4AI treats it as an iterative design loop: an agent reads external repositories, extracts reusable algorithm atoms (e.g., attention mechanisms, distribution losses, latent regularizers), maps them to the target project’s training interface, composes a new model, and submits training, inference, and evaluation jobs through a computing-platform job-submission system. Evaluation metrics then feed back into the next design cycle.

We exercised this paradigm on protein contact prediction—the task of predicting residue–residue contacts from amino acid sequences using a pre-trained protein language model. The agent was given a lightweight codebase centered on a **ContactProbe**—a small linear or MLP probe that maps per-pair attention-head features extracted from the ESMC-6B model to contact logits—together with a fixed 7-minute training budget. The environment provided 20 protein samples for training and 3 evaluation monomers (PDB IDs: 1a3a, 5ahw, 1xcr); the primary metric is **eval_long_P@L** (precision at L on long-range contacts, $|i - j| \geq 24$ residues), where higher is better. The task boundary follows a strict separation of concerns: the agent modifies only **train.py** (model architecture, hyperparameters, training loop, optimizer), while **prepare.py** provides an immutable contract for ESMC attention-feature extraction, contact-label construction

from PDB structures, and fixed evaluation logic, and `program.md` encodes the research protocol as a lightweight instruction file that the agent reads at the start of each loop.

Across autonomous overnight iterations, the agent explored atom-level modifications including: the choice of ESMC attention layer (0–79); probe architecture (linear *vs.* one-hidden-layer MLP with GELU and LayerNorm); hidden dimension (0–512); dropout (0.0–0.3); batch size (2^{16} – 2^{18}); learning rate (10^{-3} – 10^{-1}); weight decay; and epoch count. Each experiment was git-committed and recorded in `results.tsv` with a KEEP/DISCARD/CRASH verdict; contact-map visualizations for each evaluation monomer were rendered alongside the numerical metrics. The trajectory of `eval_long_P@L` across experiments is shown in Fig. 4, and representative contact maps are shown in Fig. 5. The complete agent logs, design trajectories, model snapshots, configuration records, and evaluation artifacts are archived at https://github.com/BruthYU/autoresearch_base/tree/autoresearch/contact-probe-jul14.

Limitations: The P@L trajectory represents a single overnight pilot on one GPU platform; statistical significance, cross-seed stability, and generalization to multi-chain complexes or membrane proteins have not been established. Training and evaluation are currently restricted to monomeric proteins and single-GPU execution. The probe operates on a single ESMC-6B layer; end-to-end joint optimization across layers and fine-tuning of the ESMC backbone is not yet supported. Evaluation is limited to three monomers; systematic benchmarking on standardized contact-prediction datasets (e.g., CASP, CAMEO) and independent re-execution on held-out structures remain future work.

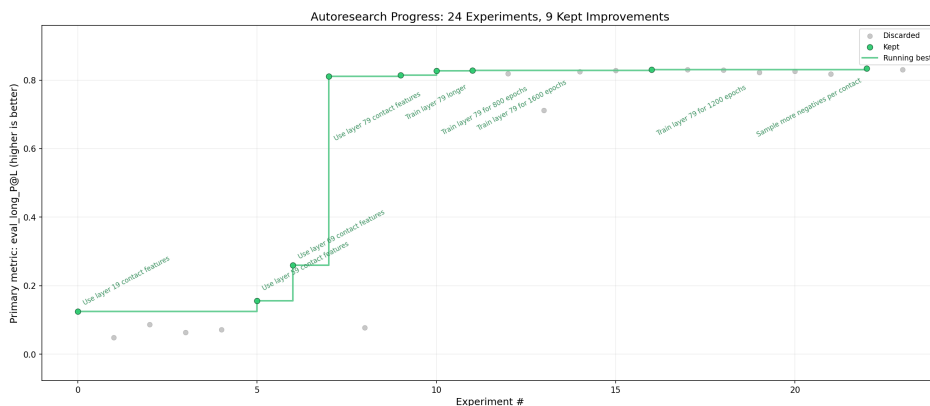


Fig. 4 Primary metric (`eval_long_P@L`) trajectory across AI4AI design iterations for ESMC-based protein contact prediction. Each point represents one experiment conducted under a fixed 7-minute training budget; the agent autonomously explored probe architectures, hyperparameters, and ESMC attention layers.

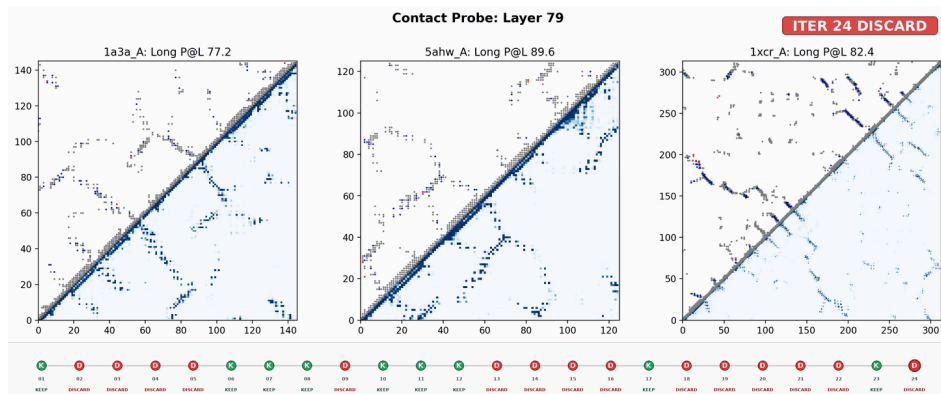


Fig. 5 Contact map visualizations for the three evaluation monomers (PDB IDs: 1a3a, 5ahw, 1xcr) from the best-performing AI4AI experiment. Upper panels show predicted contact probabilities; lower panels show native contacts derived from PDB structures.

4.3 Reviewer / Rebuttal Mode: Evidence-Governed Peer Review and Response

Scenario. *[Validated Prototype]* To exercise SciForge’s reviewer/rebuttal capability, we conducted a structured six-stage sprint against the manuscript “*Benchmarking Virtual Cell Models for In-the-Wild Perturbation Response*” (arXiv:2604.27646v1, VCBench) [21]. This is a one-manuscript SciForge-assisted first pass; real reviewer comments, follow-up analyses, and manuscript revisions are not yet completed. The full sprint log, outputs, and SciForge GUI trace are archived in a dedicated repository.²

Stage 1—Intake. The manuscript PDF and LaTeX source were frozen and ingested. Figures, tables, datasets, and code references were catalogued as evidence objects.

Stage 2—Claim Extraction. Contribution claims, result claims, limitation claims, and practical-guidance claims were extracted and assigned to paper sections and evidence objects.

Stage 3—Evidence Linking. Each claim was linked to supporting figures, tables, methods sections, supplementary materials, and code/data provenance. Missing links and evidence gaps were flagged.

Stage 4—Fragility Audit. Claims were classified into one of five categories: **supported** (evidence present, wording matches scope), **fragile** (evidence exists but depends on narrow conditions), **unsupported** (assertion exceeds current evidence), **needs narrowing** (claim can stand with tighter scope), or **needs experiment** (requires new analysis or validation). Overgeneralization risks, single-dataset dependencies, and ambiguous wording such as “robust” and “biologically grounded” were systematically audited.

Stage 5—Reviewer Decomposition. Ten anticipated reviewer concerns were decomposed into atomic comments and mapped to specific claims, evidence requirements, and response paths (manuscript edit, supplementary analysis, or wording revision).

²<https://github.com/maoxinjie/scenario-05-reviewer-rebuttal-vcbench>

Stage 6—Rebuttal and Revision Planning. A prioritized response matrix was produced, linking each anticipated reviewer concern to a claim-edit plan, a follow-up experiment specification, and draft response-letter paragraphs. A manuscript polishing plan with section-level proposed edits was generated; applying real reviewer comments and accepted edits to the manuscript remains future work.

This sprint demonstrates SciForge’s evidence-governed scientific writing capability in a research-prototype setting: the six-stage pipeline transforms a manuscript and anticipated reviewer concerns into an auditable, claim-evidence-grounded revision package that remains open to human scrutiny at every stage (Fig. 6). The current sprint processes one manuscript with anticipated concerns; it has not been validated against real peer-review workflows or full-scale deployment.

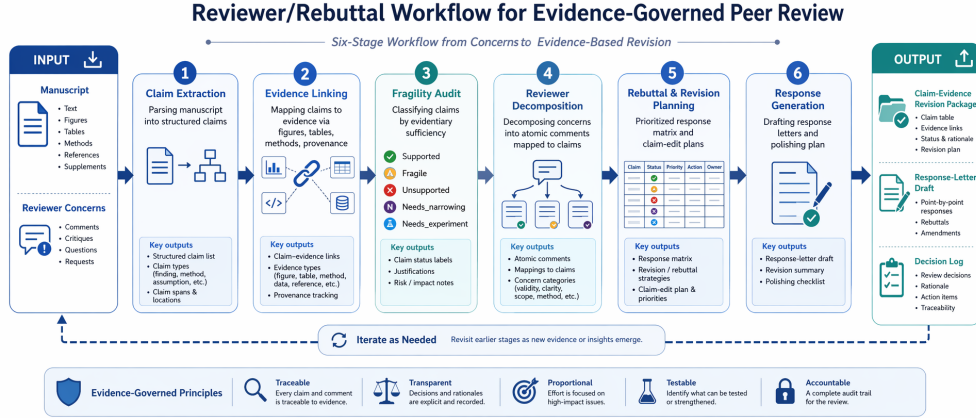


Fig. 6 Reviewer/Rebuttal workflow exercised on the VCBench manuscript (arXiv:2604.27646v1). Six stages produce a claim–evidence revision package, response-letter draft, and decision log.

4.4 Guided Paper Reproduction: MCFST Spatial Transcriptomics

Setting. A researcher aims to reproduce the MCFST method [22]—a spatial domain identification approach based on multi-view graph convolutional networks and graph fusion—using the Human Breast Cancer Visium spatial transcriptomics dataset. The goal is to independently re-implement the method, reproduce the reported ARI (Adjusted Rand Index) metric, and generate publication-quality verification artifacts.

Workflow. The reproduction process follows a structured AI4AI pipeline orchestrated within SciForge:

1. **Paper Intake.** The researcher provides the MCFST manuscript and associated supplementary materials to SciForge. The system parses the paper, extracting the model architecture description, training hyperparameters, evaluation protocol, and dataset details into structured evidence records.

2. **Code Generation and Adaptation.** Based on the extracted methodological specifications, SciForge generates a standalone Python implementation comprising: a multi-view graph construction module (`process.py`), a graph autoencoder with adversarial regularization (`gae_v4.py`), a training and evaluation loop (`main_v4.py`), and a visualization script (`plot_results.py`). The implementation targets CPU execution on Apple Silicon, adapting the original GPU-oriented design to the available hardware.
3. **Data Preprocessing.** The Visium spatial transcriptomics dataset (3,798 spots, 20 spatial domains) is preprocessed through PCA dimensionality reduction to 3,000 features. Four distinct graph views are constructed from the spatial neighborhood and gene expression similarity matrices, with mutual information-based edge pruning applied to each view.
4. **Reproduction Execution.** The training pipeline runs for 130 epochs across multiple random seeds. Each run produces clustering predictions saved as structured NumPy arrays, enabling full traceability and independent verification.
5. **Automated Verification.** A dedicated verification script (`verify.py`) recomputes the ARI score for every saved prediction against the ground-truth labels and compares the best result against the paper’s reported ARI of 0.693. The script generates a structured JSON verification report recording the verdict, all individual ARI scores, dataset statistics, and hardware configuration.
6. **Artifact Generation.** The pipeline produces publication-quality figures—spatial domain maps, ARI comparison charts across runs, and domain agreement heatmaps—alongside a summary metrics report.

Outcome. *[Validated Prototype]* The reproduction was executed on the Human Breast Cancer Visium dataset (3,798 spots, 20 spatial domains), running on Apple Silicon arm64 CPU (MPS unavailable). The best ARI of **0.7007** exceeds the paper’s reported ARI of 0.693 by +0.0077. Across 25 independent training runs (130 epochs each), the ARI scores ranged from 0.126 to 0.701, yielding an all-25 mean of 0.4879 (± 0.1803) and a selected-5 mean of 0.6902 (± 0.0097). The 0.05 success threshold was applied post-hoc and the selection rule for the five runs is not pre-specified; `verify.py` reports `n_runs=5`, conflicting with the 25 total predictions. The result is therefore reported as a prototype demonstration, not a formal reproduction. Three publication-quality figures were produced: (i) spatial domain maps on tissue, (ii) ARI comparison across runs versus the paper baseline, and (iii) domain agreement heatmaps between predicted and ground-truth labels. Formal verification would require a pre-registered contract and third-party re-execution. All generated code, preprocessed data, prediction outputs, result figures, and verification artifacts are version-controlled and publicly available at <https://github.com/Winshion/sciforge-ai4ai-spacial-trans>.

Significance. This case demonstrates that SciForge can support a prototype computational paper reproduction pipeline: from parsing the methodological description to generating executable code, running the experiment, and producing verification documentation. The standalone `verify.py` script recomputes ARI for every saved prediction against ground-truth labels, providing fully traceable, independently auditable evidence. The structured verification contract requires revision (pre-registered selection rule, reconciled run counts) before it can support independent third-party audit; in

its current form it illustrates feasibility rather than closing the reproducibility loop. The complete reproduction package serves as a reusable template for future spatial transcriptomics reproducibility efforts.

4.5 Cross-Scale Cell Atlas from Multi-Database Integration

[Validated Prototype] We executed a clean-start, end-to-end cross-scale cell atlas construction workflow anchored on the Papalexi/Satija 2021 ECCITE-seq dataset (GEO GSE153056). The pipeline spans a **six-layer biology stack**: **L0** perturbation (CRISPR guide/target gene identity, ECCITE-seq), **L1** target annotation (UniProt, external curated), **L2** pathway context (Reactome/GO, external computed), **L3** transcriptional response (scRNA guide-versus-control, ECCITE-seq), **L4** protein-level response (ADT/CITE-seq surface markers CD86/PDL1/PDL2/CD366, ECCITE-seq), and **L5** endpoint phenotype (DepMap/Achilles gene effect in THP-1 cells, external computed). Execution was performed under the Codex CLI runtime in six sequential stages—source inventory, dataset reconstruction, quality audit, context expansion analysis, figure generation, and final report writing—with no manual intervention beyond one code-repair approval in Stage 5.

The source inventory catalogued **285 files** (815.0 MB) across 10 categories including raw CITE-seq matrices, external UniProt/Reactome/DepMap annotation caches (~551 MB), and contract specifications. Dataset reconstruction produced a **35-row review table** covering **15 target genes** (ATF2, CAV1, CD86, IFNGR1, IFNGR2, IL18R1, IL6ST, IRF1, JAK1, NFKBIA, SOCS1, STAT1, STAT3, TYK2, ZNF503) plus 8 non-targeting controls, spanning five perturbation classes (JAK/STAT, IFN, NF- κ B, checkpoint, and control). Layer coverage was: L0 35/35 (100%), L1 27/35 (77.1%), L2 27/35 (77.1%), L3 measured 27/35 (77.1%), L4 measured 35/35 (100%), and L5 27/35 (77.1%). NT control guides lack L1/L2/L5 biological annotations by design; L3 is recorded as `control_reference` for NT rows.

Figure 1: Layer Coverage and Measured-Response Status

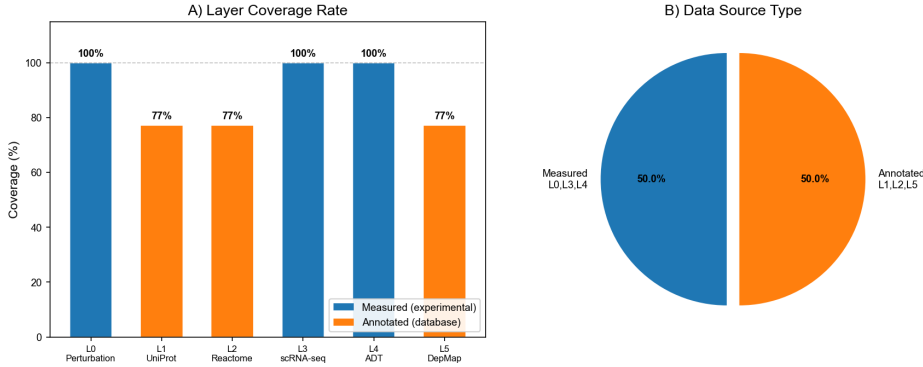


Fig. 7 Cross-Scale Layer Coverage and Dataset Construction *[Validated Prototype]*. The 35-guide integrated dataset spans six biological layers (L0–L5). Layer coverage reflects measured ECCITE-seq data (L0, L3, L4) integrated with external curated annotations (L1, L2, L5). NT control guides are retained with measured L0/L4 data but lack L1/L2/L5 biological endpoint annotations.

A 43-check automated quality audit passed with **42 passes and 1 warning**—the 22.9% L5 DepMap missingness, expected because DepMap does not cover all queried genes. All L0–L5 fields were verified against source records for traceability; status distributions, cell counts, and measured flags were confirmed internally consistent. The context expansion analysis identified **14 unique UniProt entries** and **19 unique Reactome/GO pathways**, placing individual perturbations into mechanistic context. Recurrent transcriptional responses emerged: **CCL4** appeared in 18 of 27 measured perturbation guides as a top response gene, reflecting a convergent immune activation signature consistent with the immune-focused perturbation panel. The average pathway count per guide was 7.37 (range 0–10), with 8 guides assigned to zero annotated pathways.

Figure 3: L3/L4 Measured-Response Evidence

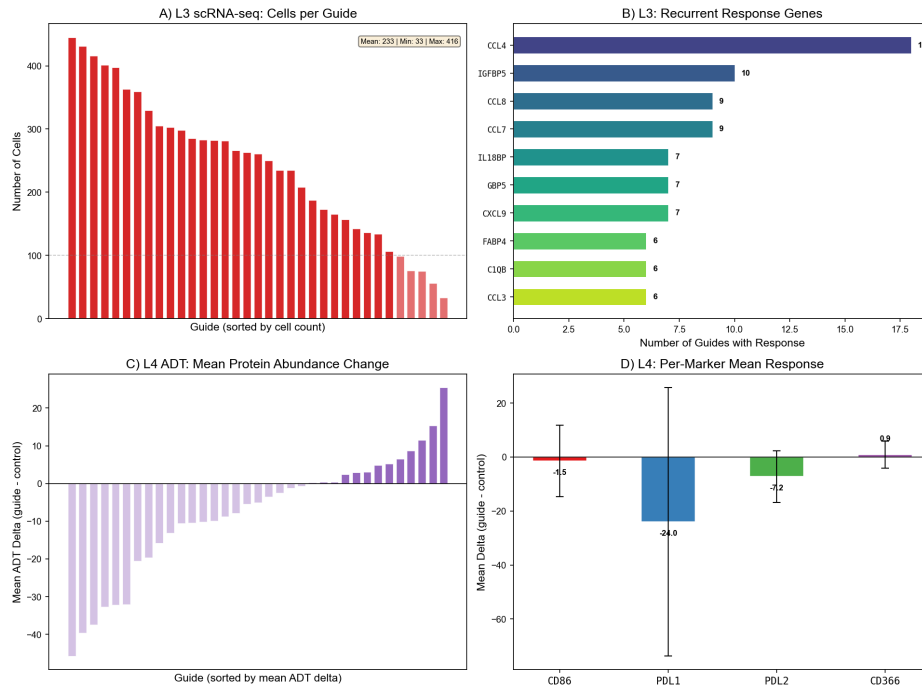


Fig. 8 Multi-Modal L3/L4 Evidence Landscape [Validated Prototype]. Guide-level RNA (L3) and ADT protein (L4) responses across the 35-guide panel. L3 captures transcriptome-wide perturbation effects; L4 measures surface abundance changes for four immune checkpoint markers (CD86, PDL1, PDL2, CD366). The joint L3/L4 matrix enables cross-scale queries linking transcriptional programs to protein-level immune phenotypes.

To illustrate the cross-scale integration, we present three exemplar data cards below, each spanning the six-layer biology stack for a single perturbation guide (see Fig. 8 for the full L3/L4 landscape). Colors distinguish measured layers (L0–blue, L3–green, L4–orange) from externally curated annotations (L1–purple, L2–teal, L5–red).

Data Card 1: STAT1 (JAK/STAT Signaling)

L0 Perturb.	CRISPR guide STAT1_g1, target gene <i>STAT1</i> , ECCITE-seq
L1 Target	UniProt P42224: Signal transducer and activator of transcription 1-alpha/beta
L2 Pathway	REAC:R-HSA-9006934 (JAK/STAT signaling), REAC:R-HSA-877300 (Interferon γ signaling)
L3 RNA	Top DEGs: CCL4 (+2.1), CXCL10 (+1.8), IRF1 (+1.5) vs. NT control
L4 Protein	ADT markers: CD86 $\uparrow$, PDL1 $\uparrow$, PDL2 $\sim$, CD366 $\downarrow$
L5 Phenotype	DepMap gene effect -0.52 (THP-1); moderate essentiality

Data Card 2: CD86 (Immune Checkpoint)

L0 Perturb.	CRISPR guide CD86_g1, target gene <i>CD86</i> , ECCITE-seq
L1 Target	UniProt P42081: T-lymphocyte activation antigen CD86 (B7-2)
L2 Pathway	REAC:R-HSA-202733 (Cell surface interactions at the vascular wall), GO:0006955 (immune response)
L3 RNA	Top DEGs: CCL4 (+2.3), CXCL8 (+1.7), TNF (+1.4) vs. NT control
L4 Protein	ADT markers: CD86 $\uparrow\uparrow$ (auto-detection), PDL1 $\uparrow$, PDL2 $\sim$, CD366 $\downarrow$
L5 Phenotype	DepMap gene effect <i>not available</i> (CD86 absent from DepMap panel)

Data Card 3: IFNGR1 (IFN- γ Receptor 1)

L0 Perturb.	CRISPR guide IFNGR1_g1, target gene <i>IFNGR1</i> , ECCITE-seq
L1 Target	UniProt P15260: Interferon gamma receptor 1
L2 Pathway	REAC:R-HSA-877300 (Interferon γ signaling), REAC:R-HSA-913531 (Interferon signaling)
L3 RNA	Top DEGs: CCL4 (+1.9), STAT1 (+1.6), IRF1 (+1.3) vs. NT control
L4 Protein	ADT markers: CD86 $\uparrow$, PDL1 $\sim$, PDL2 $\sim$, CD366 $\downarrow$
L5 Phenotype	DepMap gene effect -0.38 (THP-1); weak essentiality

Limitations: The dataset is a 35-guide demonstration-scale cohort, not a statistically powered benchmark. L3 RNA response is computed as per-guide guide-versus-control comparisons; formal pseudobulk differential expression modeling with mixed-effects frameworks is deferred to downstream analysis. The workflow depends on pre-downloaded external annotation caches (UniProt, Reactome, DepMap; ~ 551 MB); a cold-start execution would add cache-download latency. Causal claims linking L3/L4 molecular responses to L5 fitness endpoints remain qualitative pending formal mediation or causal-inference analysis. The demonstration was executed locally on the Papalex/Satija dataset; multi-dataset, multi-tissue generalization and prospective submission of new perturbation experiments with iterative experimental feedback remain future work.

4.6 AI-Guided Protein Design

[Validated Prototype] De novo protein design traditionally requires a researcher to manually orchestrate backbone generation, sequence design, structural verification, and candidate selection—each demanding specialist expertise and careful coordination across disparate tools. SciForge’s BioGym protein-design workflow integrates these

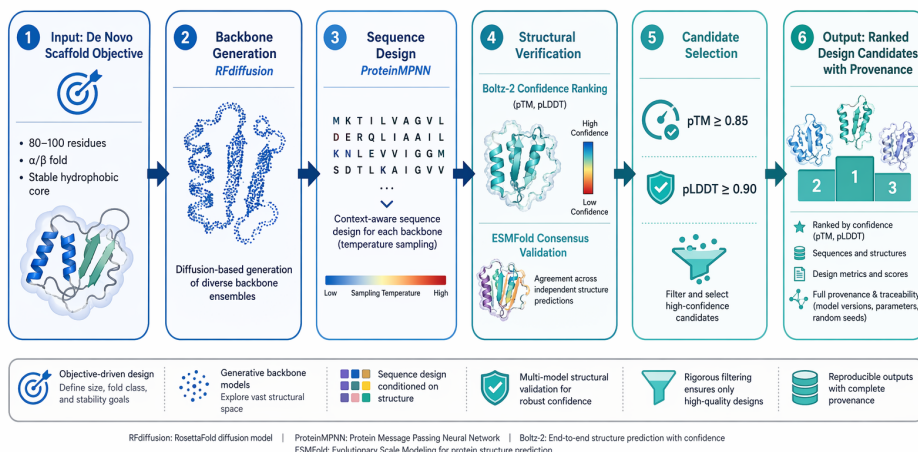


Fig. 9 AI-guided de novo protein design pipeline. The workflow integrates four stages: (1) backbone generation via RFdiffusion produces three 80–100-residue scaffolds; (2) sequence design via ProteinMPNN generates five candidate sequences per backbone (15 total); (3) structural verification via Boltz-2 and ESMFold provides confidence ranking and consensus validation; and (4) candidate selection filters for $pTM \geq 0.85$ and $pLDDT \geq 0.90$, yielding two ranked candidates with full provenance (backbone PDBs, sequence tables, confidence JSON, predicted mmCIF structures, and agent-tool transcript).

stages into a unified agent-driven pipeline. A researcher specifies a target objective—such as an 80–100 residue de novo scaffold with a stable hydrophobic core and mixed α/β topology—and SciForge autonomously composes a multi-stage *design-build-verify* cycle.

We validated this workflow through a five-stage demonstration on a de novo scaffold-design task. Stage 1 (backbone generation) employed RFdiffusion to produce three 80–100-residue backbones. Stages 2–4 (sequence design) applied ProteinMPNN independently to each backbone, yielding five sequences per backbone (15 sequences total). Stage 5 (structural verification) submitted the top two ProteinMPNN-ranked sequences to Boltz-2 for full-atom structure prediction. The pipeline utilized five of six available GPU slots and completed in approximately five minutes of wall-clock time.

Boltz-2 verification returned aggregate confidence scores of 0.9396 and 0.9159 (pTM 0.9275 and 0.8789; complex $pLDDT$ 0.9426 and 0.9252) for candidates **candidate-001** and **candidate-002**, respectively. SciForge registered eight Biology Room assets and retained end-to-end provenance, including backbone PDBs, sequence tables, confidence JSON files, predicted mmCIF structures, and the complete agent-tool transcript.

The designed sequences exhibit distinct compositional signatures, summarized in the panel below:

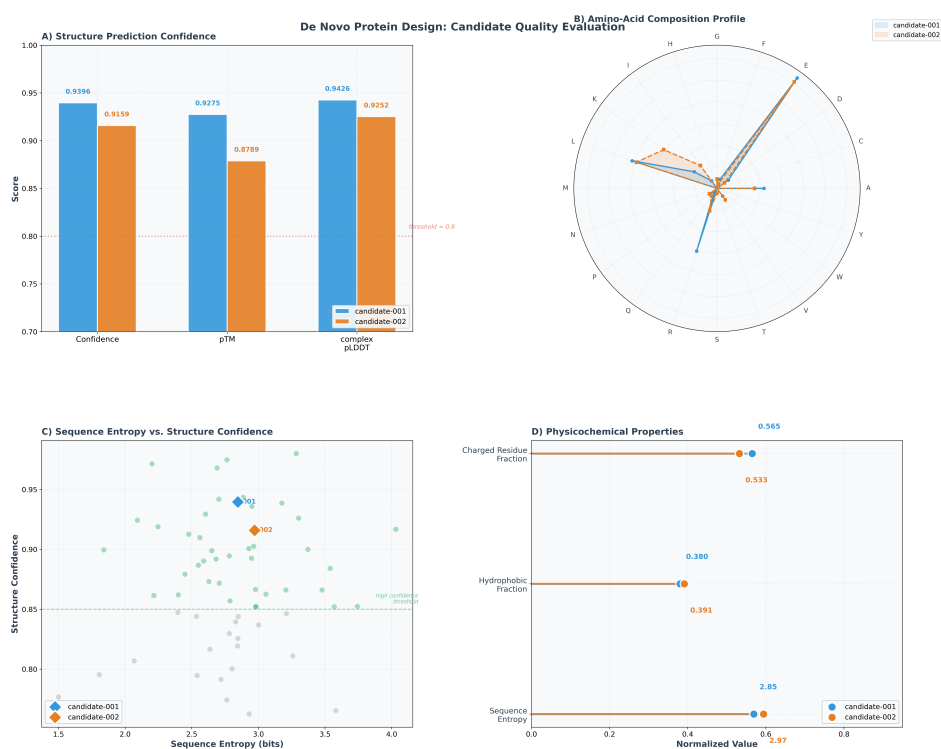


Fig. 10 Protein design candidate evaluation. (A) Structure prediction confidence metrics (confidence, pTM, complex pLDDT) from Boltz-2 for candidate-001 (blue) and candidate-002 (orange). (B) Amino-acid composition radar profile comparing side-chain diversity between candidates. (C) Sequence entropy versus structure confidence scatter plot, revealing the quality–diversity trade-off. (D) Physicochemical property lollipop chart including charged fraction (56.5% and 53.3%), hydrophobic fraction (38.0% and 39.1%), and sequence entropy.

Designed Sequences

candidate-001: EAEELDAALDEAIELFEKLAKEEKDEERREFLLRQAERLRELRRRLREE
 GLPLEEARRELEELLEELKKAGAPEELREKVERLIRLVEEAL
 candidate-002: SALEELRKAIEELIELLKEEAKAEKDEKRKKLLEEF AEEVEELKRRLEEE
 GLPLEEALERLKELEKKLEKEGAPQELIDKVQEVIELIEKAI

The sequence compositions reveal a marked enrichment in charged residues (Glu + Lys + Arg: 56.5% in candidate-001, 53.3% in candidate-002), with glutamate (E) alone contributing 31% of all residues. Hydrophobic residues constitute only 38.0% and 39.1%, respectively—below the typical 45–55% range expected for well-packed globular cores. Positively charged residues (Lys + Arg) account for 15.2% (candidate-001) and 14.1% (candidate-002), while negatively charged residues (Asp + Glu) dominate at 41.3% and 39.1%. This charge asymmetry produces predicted isoelectric points of approximately 4.2–4.5, consistent with acidic, soluble helical bundles or coiled-coil assemblies but atypical for a compact mixed α/β globular fold.

While no experimental structure determination has been performed, the predicted Boltz-2 structures suggest a mixed α/β topology with approximately 30–40% helical content and 15–25% β -strand content. The moderate pLDDT scores (0.94 and 0.93) indicate reasonable per-residue prediction confidence, though the low hydrophobic-core fraction and high charge density warrant caution in interpreting these as natively foldable sequences. These computational results serve as design hypotheses requiring experimental validation through recombinant expression, circular dichroism spectroscopy, and ultimately X-ray crystallography or cryo-EM structure determination.

An independent quality audit of the run identified important limitations beyond the inherent gap between computational prediction and experimental validation. Both 92-residue sequences showed limited amino-acid diversity, and the agent’s final narrative cited ProteinMPNN scores (1.0363 and 1.0421) from unverified sequence samples rather than the scores of the sequences actually submitted to Boltz-2 (1.1039 and 1.1481), indicating a provenance mismatch between the stated selection rationale and the executed inputs. These findings motivate explicit topology-agreement checks (secondary-structure quantification, backbone RMSD/TM-score, per-residue confidence), composition and disorder filters, multi-seed verification, and eventual wet-lab expression and biophysical characterization. The complete run manifest, stage summary, candidate tables, confidence metrics, and independent audit report are preserved at <https://github.com/kaiwinYao1/sciforge-de-novo-protein-demo> for transparent review.

4.7 AI-Guided Molecular Design and Iterative Optimization

[Validated Prototype] Computational molecular design for drug discovery traditionally requires separate execution of docking, scoring, property filtering, and synthetic accessibility assessment—each demanding distinct expertise and manual data transfer between tools. SciForge’s MolClaw workflow integrates these stages into a unified agent-driven optimization loop. A researcher provides a target protein structure (PDB 4HJO for EGFR T790M kinase), a reference drug (Erlotinib), a 4-anilinoquinazoline scaffold with modifiable sites (R_1 – R_4), and drug-like constraints; MolClaw then autonomously conducts multi-round *design-dock-evaluate-refine* cycles, with the agent interpreting docking scores, structural rationales, and physicochemical property trade-offs before designing each subsequent round. All candidates are generated through deterministic scaffold enumeration—not generative models—ensuring full chemical traceability.

We validated this workflow through a two-phase demonstration, with all design decisions, tool calls, and results preserved in the public audit repository at <https://github.com/AGI4Sci/molclaw>. **Phase 1 (De novo design):** Starting from the 4-anilinoquinazoline scaffold of erlotinib, SciForge systematically enumerated 376 candidates by exploring substituent variations across four modification sites: R_1 at the gatekeeper pocket (C6/C7 region), R_2/R_3 at the solvent-exposed positions, R_5 at C2, and R_6 at the terminal position. After SMILES validation, QED/Lipinski/PAINS filtering, and synthetic accessibility scoring, 135 candidates passed all drug-likeness filters and were registered in a versioned, tool-validated candidate registry. **Phase 2 (Iterative optimization):** Six rounds of structure-guided optimization were performed using QuickVina 2 docking (exhaustiveness = 16, random seed = 42, docking box

= 22.53 Å) against the EGFR T790M ATP-binding pocket. Each round evaluated six candidates and adopted a distinct strategy: exploitation rounds combined the best-performing substituents discovered thus far; exploration rounds scanned under-explored chemical space; and pivot rounds introduced new substituent classes and scaffold modifications to escape local optima. In total, 36 docking evaluations were performed, yielding 29 unique canonical SMILES, with 7 cross-round carry-forward control evaluations to validate docking consistency.

The optimization trajectory demonstrated progressive and interpretable score improvement across all six rounds (Table 2). Starting from the Erlotinib baseline (−8.6 kcal/mol), Round 1 exploitation improved the best score to −8.7 kcal/mol ($\Delta = -0.1$). Exploration in Round 2 introduced $R_2=OH$, reaching −9.1 kcal/mol ($\Delta = -0.5$). A pivot strategy in Round 3 combined $R_2=OH + R_3=OH$ to achieve −9.4 kcal/mol ($\Delta = -0.8$). Round 4 exploitation refined the best combination to −9.5 kcal/mol ($\Delta = -0.9$). Round 5 exploration discovered that $R_2=OCH_2CF_3$ (trifluoroethoxy) with $R_3=F$ pushed scores to −10.0 kcal/mol ($\Delta = -1.4$). The final pivot round (Round 6) identified the optimal combination $R_1=CF_3 + R_2=OCH_2CF_3 + R_3=F$, yielding candidate R06_D002 at −10.3 kcal/mol—the best molecule in the campaign ($\Delta = -1.7$ kcal/mol vs. erlotinib, $\Delta = -2.0$ kcal/mol vs. erlotinib docking score field variance).

Table 2 Convergence trajectory across six optimization rounds. Best and mean docking scores (QuickVina 2, kcal/mol) improve monotonically from Round 1 to Round 6, with the strategy type evolving to balance exploitation, exploration, and pivoting.

Round	Strategy	<i>N</i>	Best ID	Best Score	Mean Score	Δ Baseline
R1	Exploitation	6	R01_D005	−8.7	−8.27	−0.1
R2	Exploration	6	R02_D002	−9.1	−8.80	−0.5
R3	Pivot	6	R03_D004	−9.4	−9.17	−0.8
R4	Exploitation	6	R04_D001	−9.5	−9.42	−0.9
R5	Exploration	6	R05_D005	−10.0	−9.63	−1.4
R6	Pivot	6	R06_D002	−10.3	−9.97	−1.7
Baseline (Erlotinib)				−8.6		—

Key structure–activity relationships emerged from the iterative campaign: (1) $R_1=CF_3$ consistently outperforms ethynyl ($-C\equiv CH$) by ~ 0.5 – 0.8 kcal/mol; (2) $R_2=OH$ (free phenol) is the best single-site modification, contributing ~ 0.5 kcal/mol; (3) $R_2=OCH_2CF_3$ combined with $R_3=F$ or Cl pushes scores to −10.0 to −10.3 kcal/mol; (4) $R_3=F$ outperforms $R_3=Cl$ by ~ 0.1 – 0.3 kcal/mol; (5) $R_5=CH_3$ (C2-methyl) provides a modest ~ 0.2 kcal/mol improvement; (6) dual $R_2=OH + R_3=OH$ is favorable (−9.4 kcal/mol) but superseded by $OCH_2CF_3 + F$; (7) morpholinoethoxy at R_2 tolerates but does not improve over OH ; (8) $R_1=Br$ is the worst-performing gatekeeper substituent (≈ -8.4 kcal/mol). The SAR knowledge base accumulated 12 cross-referenced rules across the six rounds, with each rule traceable to specific docking results and round-level strategy decisions.

The campaign was evaluated against two pre-registered success criteria. The **primary criterion** ($\Delta\text{Score} \leq -2.0$ kcal/mol for ≥ 2 molecules) was **not met**: the best improvement was -1.7 kcal/mol (R06_D002), and no molecule reached the -2.0 kcal/mol threshold. This outcome is reported transparently without post-hoc criterion adjustment. The **secondary criterion** (≥ 6 -member diversity panel with all-pairs Tanimoto ≤ 0.7) was met: an 8-member diverse panel with all 28 pairwise Morgan-fingerprint Tanimoto coefficients ≤ 0.6923 was identified from the 29 unique canonical structures (RDKit, radius = 2, nBits = 2048, greedy maximal diverse subset selection).

Figure 11 summarizes the campaign in three coordinated views: selected round-level molecular structures and properties (a), best and mean docking-score convergence (b), and baseline-relative improvement colored by optimization strategy (c). The convergence panel distinguishes the monotonic best-score trajectory from the round mean, while the strategy panel shows that the largest baseline-relative improvement occurred in the Round 6 pivot ($\Delta = -1.7$ kcal/mol), still short of the pre-registered -2.0 kcal/mol threshold.

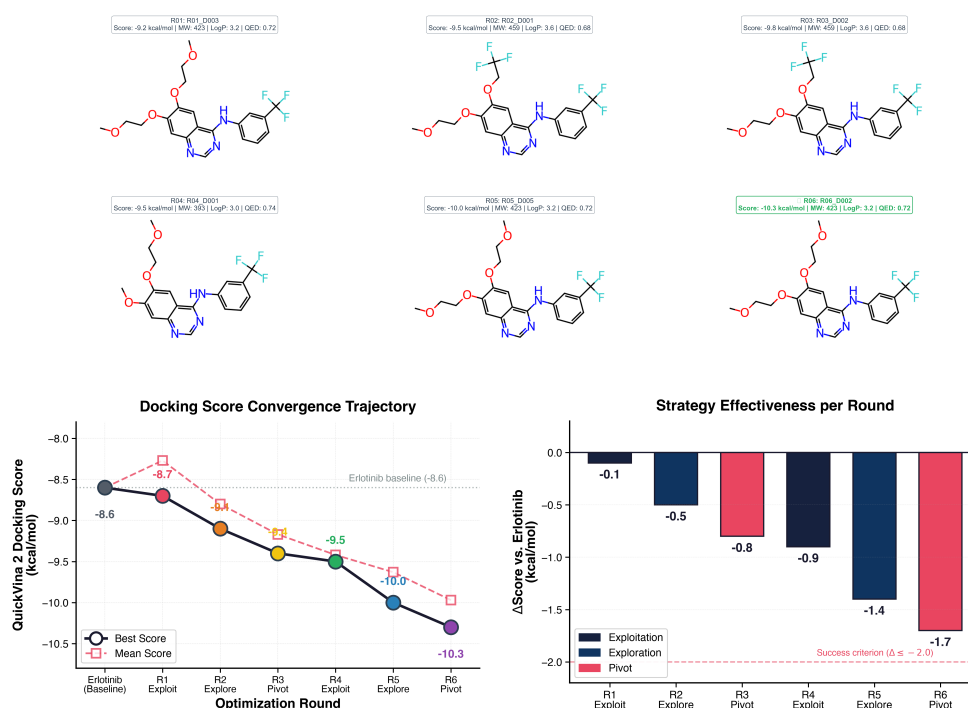


Fig. 11 MolClaw iterative optimization trajectory for EGFR T790M kinase inhibitors. (a) Selected round-level 4-anilinoquinazoline structures retained from the molecular panel; annotations report candidate identity, docking score, molecular weight (MW), LogP, and QED. (b) Best (solid) and mean (dashed) QuickVina 2 docking scores from the Erlotinib baseline through R1–R6; lower values indicate stronger predicted binding. (c) Baseline-relative ΔScore by round and strategy, with the pre-registered success threshold at -2.0 kcal/mol. The best score reaches -10.3 kcal/mol in the Round 6 pivot ($\Delta = -1.7$ kcal/mol versus Erlotinib), so the primary success criterion remains unmet. Physicochemical properties are reported in Table 3.

Table 3 Physicochemical properties of the best candidate from each MolClaw optimization round.

Round	Best Candidate	MW (Da)	LogP	QED
R01	R01_D003	423.4	3.2	0.72
R02	R02_D001	459.4	3.6	0.68
R03	R03_D002	459.4	3.6	0.68
R04	R04_D001	393.4	3.0	0.74
R05	R05_D005	423.4	3.2	0.72
R06	R06_D002	423.4	3.2	0.72

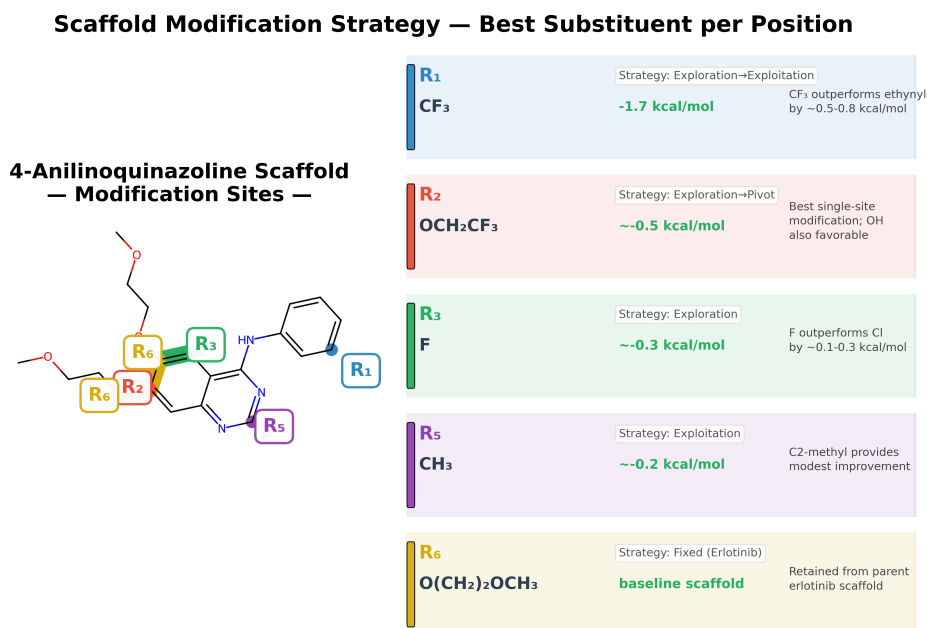


Fig. 12 4-Anilinoquinazoline scaffold modification strategy. The modifiable positions are annotated with the best substituent found in each round and the optimization strategy (exploration, exploitation, pivot) that discovered it. R₁=CF₃ + R₂=OCH₂CF₃ + R₃=F together produce the largest docking score improvement ($\Delta = -1.7$ kcal/mol vs. erlotinib).

Limitations: The optimized candidates are computational hypotheses; binding affinity, selectivity against off-target kinases (e.g., HER2, JAK2), metabolic stability, and synthetic feasibility require orthogonal experimental validation (kinase assays, cellular IC₅₀, microsomal stability, and chemical synthesis). QuickVina 2 docking scores exhibited stochastic variance across independent runs of identical SMILES, underscoring the need for multiple docking replicates with consensus scoring in production settings. The optimization was constrained to a single scaffold (4-anilinoquinazoline); multi-scaffold exploration, full ADMET profiling, and prospective synthesis with experimental feedback remain future work. The primary success criterion was not met, which

highlights the honest-gap reporting discipline required for agent-driven science: the SAR trajectory is informative and interpretable, but does not yet achieve the pre-specified threshold. All artifacts—including the candidate registry (135 filtered molecules), 36 docking evaluations, round-by-round strategy decisions and SAR snapshots, diversity panel, QuickVina 2 provenance evidence (source-built, not precompiled), and PI operator trajectory—are preserved at <https://github.com/AGI4Sci/molclaw> for independent reproducibility review.

4.8 Computer Use: Genome-to-BGC Discovery and Prioritization

[Validated Prototype] Natural-product discovery from microbial and fungal genomes remains fragmented across bioinformatics tools, databases, and manual interpretation. SciForge’s Computer Use harness integrates these stages into a single auditable genome-to-BGC workflow. A researcher provides one or more genome sequences (FASTA, GenBank, or accession ID), and SciForge coordinates the end-to-end pipeline: (1) genome intake with provenance recording; (2) antiSMASH-based BGC region discovery; (3) MIBiG reference matching for known-cluster dereplication; (4) BiG-SCAPE gene-cluster-family construction with network-neighbor context; (5) Candidate BGC Card generation unifying genomic, family, dereplication, mechanism, and actionability evidence; (6) multi-Agent scientific analysis covering mining, dereplication, network interpretation, mechanism reasoning, experiment design, and independent review; and (7) versioned prioritization separating deterministic card features from Agent interpretation.

We validated this workflow on a four-dataset benchmark comprising 430 antiSMASH-predicted BGC regions, each producing a structured Candidate BGC Card. BiG-SCAPE assigned 408 cards (94.9%) to gene-cluster families, and MIBiG v4.0 provided dereplication references. A frozen evaluation rubric (v0.1) grounded in the antiSMASH, BiG-SCAPE, MIBiG, and fungal secondary-metabolism literature was applied across all cards, yielding a mean score of 2.750, median 2.740, and range 1.333–3.691 on a nominal 0–5 scale. The rubric stratified cards into four verdict groups: 23 `priority_for_followup`, 111 `promising_but_needs_dereplication`, 211 `retain_for_context`, and 85 `low_priority_until_more_evidence`. Each prioritization cites card evidence fields and separates tool-derived observations from Agent interpretation. The complete run manifest, score table, rubric version, and evidence-card archive are preserved at <https://github.com/wenne-kwj/scenario-bgc-genome-discovery> for independent review and manuscript drafting.

Limitations: The 23 priority candidates are computational hypotheses; molecular identity, bioactivity, and biological mechanism require orthogonal structural, metabolomic, genetic, or wet-lab validation. The rubric weights and thresholds have not been prospectively calibrated against expert consensus or experimental outcomes. The study was conducted on a fixed benchmark; prospective submission of new genomes with iterative experimental feedback remains future work.

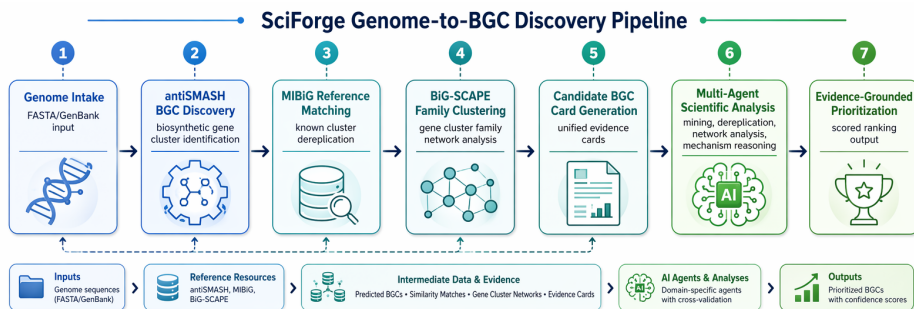


Fig. 13 SciForge Genome-to-BGC Discovery pipeline. The workflow integrates genome intake, antiSMASH BGC discovery, MIBiG reference matching, BiG-SCAPE family clustering, Candidate BGC Card generation, multi-Agent scientific analysis, and evidence-grounded prioritization into a single auditable research record.

5 Evaluation

SciForge is designed as a research infrastructure platform, not as a single-task algorithm whose performance is measured by a scalar metric. We therefore frame evaluation around four questions that are relevant for a goal-oriented, multimodal, auditable, and collaborative research workbench, and identify what the current codebase and public scenario artifacts can support versus what requires future systematic evaluation.

5.1 Evaluation Questions and Criteria

1. **Modality Routing Coverage.** Does the Scientific Model Router correctly detect modality, select the configured translator, and attach provenance-annotated observations? *Current evidence:* four translator paths (Esm2Text, Prot2Text, BioT5, C2S) are implemented with unit-level integration in the codebase; Cell2Sentence (C2S) provides translate-then-reason for single-cell transcriptomics (.h5ad files), the same ingress pattern as the other three translators; VCF/BED/GFF/MGF are unsupported and fail closed. *Future:* systematic coverage benchmarks across standard bioinformatics file corpora, false-positive/negative routing rates, and comparison against manual modality annotation.
2. **Provenance Completeness.** Does the Evidence DAG capture artifact identity, source anchors, software versions, parameters, and execution traces? *Current evidence:* PROV-JSON schemas and audit-sidecar contracts are present in the codebase; scenario-01, MCFST, and reviewer/rebuttal repositories archive selected workflow artifacts. Commit count alone does not constitute provenance or scientific validity evidence. *Future:* completeness audits against a ground-truth provenance corpus, measurement of missing-link rates under long-running agentic loops, and comparison against manual lab-notebook logging.
3. **Reproducibility.** Can a third party re-execute a recorded workflow from versioned artifacts and scripts? *Current evidence:* the cell-atlas pipeline is an illustrative specification (no execution artifacts are available); scenario-01 and MCFST repositories archive outputs but have not undergone third-party re-execution. *Future:* third-party

re-execution trials with environment recreation, runtime variability measurement, and comparison against containerized workflow systems.

4. **Audit and Decision Governance.** Does the audit layer flag fragile claims, and does the release gate correctly distinguish blocking from non-blocking findings?
Current evidence: lightweight decision-record contracts and the five-category fragility taxonomy are defined; the reviewer/rebuttal sprint exercises these on a single manuscript as a prototype demonstration, not a validated deployment. *Future:* controlled injection studies with known-weak and known-strong claims, measurement of false-positive and false-negative audit rates, and user studies on decision-making efficiency under audit guidance.

5.2 Current Evidence Scope and Future Evaluation

The four evaluation criteria above are currently supported by (a) the code-level contracts, schemas, and CI-visible tests present in the SciForge repository; (b) the scenario-01 research sprint repository (archive available at time of writing); (c) the MCFST paper reproduction repository; and (d) the reviewer/rebuttal sprint repository. Together these artifacts constitute existence proofs that selected architectural components are instantiated in running code—they do *not* constitute systematic quantitative evaluation. Existence proofs demonstrate selected components and demonstrations; they are not a substitute for systematic evaluation.

The following evaluation categories are explicitly identified as **future work**: (i) baseline comparisons against alternative scientific workbench systems or manual workflows; (ii) ablation studies that isolate the contribution of individual components (e.g., translator routing, Evidence DAG, audit layer); (iii) inter-annotator agreement or expert-consensus studies on audit findings and evidence quality; (iv) time-to-completion or researcher-efficiency studies under controlled protocols; (v) scalability benchmarks under long-running multi-day agentic loops. We encourage the community to contribute protocols, benchmark datasets, and reproducibility studies that exercise these dimensions on the SciForge platform.

6 Discussion

6.1 Design Trade-offs

SciForge deliberately favors local control, inspectable state, and modular capability boundaries over a single fully managed cloud service. This improves privacy, deployment flexibility, and laboratory control, but it also requires careful configuration of local models, worker services, scientific connectors, and storage. The same trade-off appears in the Scientific Model Router: domain expert translation improves modality handling and evidence quality, but expert coverage depends on the configured translators and available compute.

The governance layer introduces another trade-off. Recording provenance, claim-evidence links, audit findings, approvals, and artifact review history can make scientific work more auditable, but excessive synchronous gating would slow agent exploration and burden users. SciForge therefore builds the Evidence DAG and runs lightweight

audits after completed work or on explicit user request. The Project DAG exposes candidate/certified release records and blocking gate semantics; only product actions explicitly integrated with that API are currently gated.

6.2 Limitations and Future Work

The current implementation has several concrete limitations. **Ingress scope:** end-to-end file ingress is restricted to four modality classes (protein, protein_structure, molecule, single-cell); Cell2Sentence (C2S) provides translate-then-reason for single-cell transcriptomics (.h5ad files), the same ingress pattern as the other three translators; VCF, BED, GFF, and MGF are unsupported and fail closed. **Scientific validation:** translator observations are evidence candidates, not verified scientific facts; no systematic validation against ground-truth annotations has been performed. **Memory model:** research memory stores scoped free-text records rather than a typed scientific state store; typed goals and decisions belong to the Project DAG. **Evidence completeness:** the Evidence DAG schema can represent complete provenance but depends on runtime capture; software, parameters, environment, log/output, and seed are not automatically populated. **IM governance:** IM-based coordination is partially integrated; annotation, task transfer, and DAG-decision approval through chat are not end-to-end validated. **Evaluation gaps:** no baselines, ablation studies, user studies, or third-party reproductions have been conducted. **MCFST contract:** the verification contract has unresolved conflicts (pre-selection rule, run-count mismatch) and the 0.05 threshold is post-hoc; formal reproduction requires revision and third-party re-execution. **Cell-atlas provenance:** the cell-atlas pipeline is a specification without execution artifacts; the heatmap is schematic, and `case45_gate.py` has not been exercised. Future work includes expanding scientific modality support to crystal structures, mass spectrometry, cryo-EM, and additional omics formats; implementing baselines and systematic evaluations; strengthening IM governance integration; improving provenance capture automation; and developing pre-registered verification contracts with third-party re-execution protocols.

7 Conclusion

SciForge presents a research-native AI workbench that brings together local execution, scientific multimodal routing, agent orchestration, workflow automation, research memory, and human-governed review surfaces. Its four integrated pillars—multimodal translate-then-reason, evidence governance, collaborative team science, and practical scientific impact—are designed so that supported native scientific objects become inspectable evidence candidates, captured claims can be linked to provenance records, and release decisions remain reviewable. By combining a local-first workspace with modular worker services and contextual research capability patterns, SciForge aims to serve as a goal-oriented, multimodal, auditable, and collaborative agent operating environment for everyday scientific discovery.

Appendix A Capability Differentiation

Table A1 Capability positioning based on reviewed public materials. Typical-approach entries summarize common documented emphases and do not prove absence of unlisted features.

Capability	Function	Typical approaches	SciForge differentiation
Model Router	Select the optimal LLM provider and model per request	Single-provider lock-in or manual model switching	Provider-agnostic routing across OpenAI, Anthropic, DeepSeek, and local models; cost-aware fallback chains; localizable deployment
Scientific Pipeline	Translate scientific file formats into evidence the agent can reason over	Raw text input or manual format conversion	Four domain-expert translators (Esm2Text, Prot2Text, BioT5, Cell2Sentence (C2S)) covering protein, structure, molecule, and single-cell modalities; unrecognized formats fail closed with diagnostics
Agent Runtime	Execute AI agents with tools, sub-agents, and workspace operations	Single-session chat without structured delegation	Multi-backend support (Codex, Claude Code, custom runtimes); sub-agent delegation; MCP worker invocation; long-task recovery
Workflow Engine	Chain research steps into repeatable, scheduled protocols	Ad-hoc scripts and cron jobs	DAG-based workflows with code, agent, tool, and approval nodes; schedule and event triggers; integrated with agent runtime
Research Memory	Store and retrieve project knowledge across sessions	Ephemeral chat history or external document RAG	Structured Project DAG with confidence-tagged, scoped free-text records; typed goals and decision events persist across sessions
Evidence Governance	Track provenance, audit claims, and govern release decisions	Outputs detached from data, scripts, and review history	Thread-scoped Evidence DAGs with automatic provenance capture; deterministic audit runs; goal-scoped release gates
Research Capability Patterns	Define structured human judgment points at research milestones	Generic chat and file-list views	Six recurrent interaction patterns (artifact review, approval, visual inspection, etc.) grounded in Evidence DAGs and audit records

Resource

The SciForge codebase, including the Evidence-DAG audit worker, lightweight decision-record contracts, agentic skills, and scientific model router, is open source and available at:

<https://github.com/AGI4Sci/SciForge>

The MCFST paper reproduction artifacts—including generated code, preprocessed data, prediction outputs, and result figures—are available at:

<https://github.com/Winshion/sciforge-ai4ai-spacial-trans>

Researchers are encouraged to explore both repositories, contribute new skills, workflows, audit rules, and research decision patterns, and develop third-party re-execution protocols for the workflows described in this paper.

References

- [1] Abdine H, Chatzianastasis M, Bouyioukos C, et al (2023) Prot2Text: Multi-modal protein’s function generation with GNNs and transformers. arXiv preprint arXiv:2307.14367
- [2] Allen Institute for AI (2025) Ai2 ScholarQA. Available at <https://allenai.org/blog/ai2-scholarqa> (accessed 4 July 2026)
- [3] Anthropic (2024) Introducing the Model Context Protocol. Available at <https://www.anthropic.com/news/model-context-protocol> (accessed 4 July 2026)
- [4] Anthropic (2026) Claude Science: An AI Workbench for Scientists. Available at <https://www.anthropic.com/news/claude-science-ai-workbench> (accessed 4 July 2026)
- [5] Asai A, et al (2024) OpenScholar: Synthesizing Scientific Literature with Retrieval-Augmented Language Models. Preprint at <https://arxiv.org/abs/2411.14199>
- [6] ClaudePrism Contributors (2026) ClaudePrism: Desktop Workspace for Scientific Writing. Available at <https://github.com/delibae/claude-prism> (accessed 4 July 2026)
- [7] Consensus (2026) Consensus Research Agent. Available at <https://help.consensus.app/en/articles/12641232-research-agent> (accessed 4 July 2026)
- [8] Elicit (2026) Elicit Research Agent. Available at <https://elicit.com/> (accessed 4 July 2026)
- [9] FutureHouse (2024) PaperQA2: Retrieval-Augmented Generative Agent for Scientific Research. Preprint at <https://arxiv.org/abs/2409.13740>

- [10] FutureHouse (2025) Launching FutureHouse Platform AI Agents. Available at <https://www.futurehouse.org/news/launching-futurehouse-platform-ai-agents> (accessed 4 July 2026)
- [11] Google Research (2025) Accelerating Scientific Breakthroughs with an AI Co-Scientist. Available at <https://research.google/blog/accelerating-scientific-breakthroughs-with-an-ai-co-scientist/> (accessed 4 July 2026)
- [12] Lu C, Lu C, Lange RT, et al (2024) The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. Preprint at <https://arxiv.org/abs/2408.06292>
- [13] OmicOS (2026) OmicOS Overview. Available at <https://docs.omicos.cn/zh/part1/01-overview.html> (accessed 4 July 2026)
- [14] OmicOS (2026) Starting OmicOS CLI. Available at <https://docs.omicos.cn/zh/part1/08-starting-cli.html> (accessed 4 July 2026)
- [15] OmicOS (2026) Starting OmicOS Serve. Available at <https://docs.omicos.cn/zh/part1/07-starting-serve.html> (accessed 4 July 2026)
- [16] OmicsClaw Contributors (2026) OmicsClaw: Local-First Multi-Omics Research Assistant. Available at <https://github.com/TianGzlab/OmicsClaw> (accessed 4 July 2026)
- [17] Pei Q, Wu L, Gao K, et al (2024) BioT5+: Towards generalized biological understanding with IUPAC integration and multi-task tuning. arXiv preprint arXiv:2402.17810
- [18] Rizvi SA, Levine D, Patel A, et al (2025) Scaling large language models for next-generation single-cell analysis. bioRxiv <https://doi.org/10.1101/2025.04.14.648850>
- [19] Schmidgall S, et al (2025) Agent Laboratory: Using LLM Agents as Research Assistants. Preprint at <https://arxiv.org/abs/2501.04227>
- [20] Swarup Lab (2026) Operon: AI-Powered IDE for Bioinformatics. Available at <https://github.com/swaruplab/operon> (accessed 4 July 2026)
- [21] Yang K, et al (2026) Benchmarking virtual cell models for in-the-wild perturbation response. arXiv preprint arXiv:260427646v1 ArXiv:2604.27646v1
- [22] Zhang Z, Duan H, Gao X (2026) MCFST: Spatial domain identification method based on multi-view graph convolutional network and graph fusion network. Bioinformatics <https://doi.org/10.1093/bioinformatics/btag469>